ESCUELA SUPERIOR de Innovación y Tecnología

ESIT - ESTANCIA PROFESIONAL

Formato Oficial de Bitácora Semanal

Versión para seguimiento grupal y evaluación individual

Propósito del instrumento: Esta versión íntegra elementos de control grupal y trazabilidad individual para sustentar la evaluación del desempeño técnico, el cumplimiento, la participación y los aportes verificables de cada integrante del equipo.

Fase:	Fase 2 - Implementación y Configuración del WAF
Nombre del proyecto:	WAF Shield Cloud – Implementación de Seguridad Perimetral para Aplicaciones en la Nube
Equipo:	GRUPO SN-2
Tutor:	Ing. Carlos G. Rodriguez
Ciclo:	02
Semana No.:	tres Fecha (inicio - fin): 19/10/2026 - 8/11/2026

Instrucción de uso: La bitácora debe completarse semanalmente. Las evidencias deben ser concretas, atribuibles y verificables: enlaces, commits, rutas de archivos, capturas, pruebas, documentos o configuraciones. No se recomienda registrar aportes genéricos sin evidencia técnica.

Estancia Profesional
ESCUELA SUPERIOR DE INNOVACIÓN Y TECNOLOGÍA

1. Información general de la semana

Campo	Registro de la semana
Campo / Rol	Ingeniero de Seguridad Cloud Jr.
Nombre	Mauricio Martínez
Fase actual:	Uno - Análisis del entorno, riesgos y diseño
Semana No.:	Fase 2 - Implementación y Configuración del WAF (Semana 3)

Campos sugeridos: Integrantes presentes, roles activos, fase actual, entregable objetivo, acuerdos clave, riesgos iniciales y estado general de avance.

2. Actividades realizadas

Actividad	Descripción técnica	Responsable	Evidencia verificable
Instanciación de Sandbox Local	Configuración de contenedor Docker waf-sandbox (node:20-alpine) en puerto 3000 con mapeo host.docker.internal	Mauricio Martínez	Captura de consola docker ps y contenedor activo en :3000
Despliegue de Proxy WAF Local	Creación y ejecución de contenedor Nginx nginx_waf (nginx:alpine) en puerto 8080 implementando limit_req_zone y denegación en /admin	Mauricio Martínez	Archivo nginx_waf.conf y captura docker run en puerto 8080
Pruebas de Auditoría y Mitigación	Ejecución de scripts en Python (test_baseline_waf.py y test_waf_mitigated.py) para validar estrangulamiento de ráfagas en /login y rutas privadas	Mauricio Martínez	Captura de consola con respuestas empíricas HTTP 403 Forbidden
Redacción de Informe Individual	Elaboración y compilación del Informe Técnico Individual de la Fase 2 (11 páginas) e integración de bitácora y capturas	Mauricio Martínez	Documento PDF/Docx en historial de Git/Local

Actividad	Descripción técnica	Responsable	Evidencia verificable
Scripting de Despliegue GCP	Creación del script ejecutable deploy_waf_policy.sh en gcloud CLI para trasladar las reglas 1000, 2000, 3000 y 4000 a Cloud Armor	Mauricio Martínez	Código bash automatizado deploy_waf_policy.sh y Sección 6 del informe

3. Herramientas y servicios utilizados

Herramienta / servicio	Propósito	Observaciones
Docker Desktop / Engine	Aislamiento local de contenedores para probar la app Node.js (:3000) y Reverse Proxy Nginx (:8080)	Permitió validar el WAF perimetral localmente sin romper el avance en GCP
Python (requests)	Automatización de peticiones HTTP para simular ráfagas de fuerza bruta y vectores L7	Herramienta principal para generar métricas y evidencias empíricas de auditoría
Nginx (Alpine)	Proxy perimetral local para emular políticas de Rate Limiting y denegación de Cloud Armor	Implementó la directiva limit_req_zone con respuesta 403 a ráfagas en /login
Google Cloud SDK (gcloud)	Estructuración del script bash para desplegar reglas en Google Cloud Armor	Permite la transferencia directa de la solución local a la nube de GCP

4. Problemas detectados y análisis

Problema	Impacto	Causa probable	Evidencia
Aislamiento de red entre contenedores Docker en Windows (error redirigiendo Nginx a localhost:3000).	Fallo inicial de comunicación Proxy-Backend.	Nginx dentro del contenedor busca el puerto internamente.	Sección 2 del informe.
Observación académica en Fase 1 sobre pruebas conceptuales sin evidencia empírica.	Necesidad de sustentar el filtrado en consola.	Presentación inicial de arquitectura sin capturas de ejecuciones.	Capturas de pantalla ejecutando test_waf_mitigated.py.

5. Soluciones aplicadas

Solución aplicada	Problema asociado	Responsable	Evidencia
Implementación de la bandera --add-host=host.docker.internal:host-gateway al docker run de Nginx.	Aislamiento de red entre contenedores Docker en Windows (error redirigiendo Nginx a localhost:3000).	Mauricio Martínez (Ingeniero de Seguridad Cloud Jr.) / Equipo SN-2	Sección 2 del informe.
Ejecución de auditoría en consola y captura de respuestas StatusCode = 403.	Observación académica en Fase 1 sobre pruebas conceptuales sin evidencia empírica.	Mauricio Martínez / Grupo SN-2	Capturas de pantalla ejecutando test_waf_mitigated.py.

6. Estado del proyecto esta semana

Área	Estado	Comentarios
Ingeniero de Seguridad Cloud Jr.	Completado	Implementación y validación empírica del filtrado perimetral en Sandbox local (Docker/Nginx). Verificación de respuestas 403 Forbidden en ráfagas de fuerza bruta y redacción del script de automatización para Google Cloud Armor (gcloud)..

7. Plan de trabajo para la próxima semana

Tarea próxima semana	Responsable	Fase	Entregable esperado
Monitoreo y Análisis de Telemetría	Mauricio Martínez (Seguridad Cloud Jr.) / Equipo SN-2	Fase 3 - Despliegue y Monitoreo	Inspección de logs en Cloud Logging para verificar bloqueo de peticiones maliciosas y ausencia de falsos positivos.

Tarea próxima semana	Responsable	Fase	Entregable esperado
Consolidación de Evidencias Finales	Equipo SN-2	Fase 3 - Entrega Final	Informe Técnico Final integrado y demostración práctica del prototipo perimetral.
Coordinación general del avance de la Fase 3 y actualización del repositorio.	Julissa Fuentes (Líder) / Equipo SN-2	Fase 3	Tareas registradas en el backlog / repositorio actualizadas.

8. Aportes individuales verificables

Integrante	Rol	Tarea técnica específica realizada	Componente / entregable afectado	Evidencia verificable	Resultado
Mauricio Martínez	Ingeniero de Seguridad Cloud Jr.	Desarrollo de scripts de auditoría en Python (test_baseline_waf.py y test_waf_mitigated.py), configuración de Nginx WAF local y script bash de Cloud Armor.	Informe Técnico Individual (Fase 2), Sandbox local y repositorio GitHub	Archivos test_waf_mitigated.py, nginx_waf.conf, deploy_waf_policy.sh y capturas de consola en GitHub.	Verificación empírica en consola del bloqueo perimetral de ráfagas con código HTTP 403 Forbidden.

Uso obligatorio: Cada fila debe corresponder a un aporte individual atribuible. Ejemplos válidos: commit identificado, sección redactada en un documento, prueba ejecutada, configuración aplicada, endpoint implementado, consulta corregida, vista del dashboard ajustada o script de simulación mejorado.

9. Cumplimiento individual y responsabilidad

Integrante	Tarea asignada	Estado	Observación del cumplimiento	Seguimiento requerido
Mauricio Martínez	Configuración operativa del WAF local, pruebas de auditoría automatizadas y documentación del informe individual de la Fase 2.	Completado	Entregado en tiempo y forma con sustentación empírica en consola (código 403), informe de 11 páginas y script de despliegue gcloud.	Aplicar la política waf-shield-sn2-policy en el entorno real de Google Cloud Armor en la Fase 3.

10. Participación en documentos técnicos y entregables

Integrante	Documento / entregable	Sección elaborada o corregida	Tipo de aporte	Evidencia
Mauricio Martínez	Informe Técnico Individual – Fase 2	Elaboración integral (Secciones 1 a 7), arquitectura Sandbox, matriz de reglas, scripts Python/Nginx y script gcloud.	Redacción técnica, diseño de pruebas, codificación y documentación empírica.	Documento PDF/Docx compilado (11 páginas), bitácora actualizada e historial de commits en GitHub.

11. Pruebas, configuraciones o validaciones por estudiante

Integrante	Tipo de prueba / configuración	Elemento intervenido	Resultado	Evidencia
Mauricio Martínez	Prueba de estrangulamiento de tasa (Rate Limiting) y bloqueo de rutas perimetrales	Reverse Proxy Nginx en contenedor Docker (:8080) y backend Node.js (:3000).	Interceptación inmediata en el perímetro con código HTTP 403	Captura de pantalla de PowerShell ejecutando

Integrante	Tipo de prueba / configuración	Elemento intervenido	Resultado	Evidencia
	(/admin).autenticación (login) y accesos por roles.		Forbidden a partir de la 2.ª petición en /login.	python test_waf_mitigated.py e imágenes de contenedores en docker ps.

12. Participación en demo o explicación técnica (Solo utilizar en entrega del informe final)

Integrante	Parte explicada o demostrada	Evidencia presentada	Nivel de dominio observado

13. Reflexión individual por rol

Integrante	Rol	Qué realizó	Dificultad técnica enfrentada	Cómo la resolvió	Próximo paso
Mauricio Martínez	Ingeniero de Seguridad Cloud Jr	Configuración de la capa mitigada local en Nginx, desarrollo de la suite de pruebas HTTP en Python y estructuración del script para GCP.	Resolver la comunicación de red entre el contenedor Proxy Nginx y el backend de Node.js en Windows Docker Desktop.	Utilizando la bandera de red --add-host=host.docker.internal:host-gateway al instanciar el contenedor de Nginx.	Monitorear resultados de intento de ataques en Armor en la Fase 3.

14. Registro consolidado de evidencias

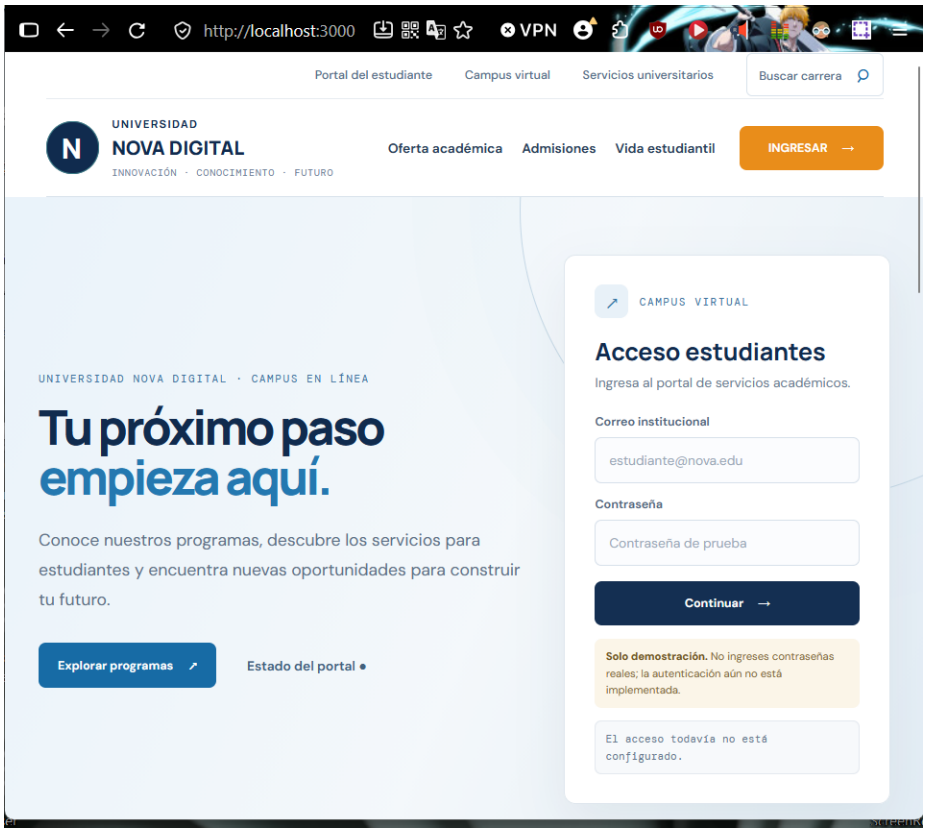
No.	Descripción de la evidencia	Tipo	Responsable	Enlace / ruta / referencia
1	Clonación local y despliegue de la aplicación web del desarrollador Jhony Jiménez para la ejecución de pruebas de auditoría de línea base en Sandbox local.	Repositorio / Código fuente	Mauricio Martínez (Seguridad Cloud Jr.) / Jhony Jiménez (Dev Web Jr.)	Repositorio GitHub / Carpeta tarea-main
2	Contenedores Docker activos	Captura de pantalla	Mauricio Martínez	Captura de ejecución en GitHub
3	Script de auditoría en Python (test_waf_mitigated.py) con la suite de pruebas HTTP	Script	Mauricio Martínez	Repositorio GitHub / test_waf_mitigated.py
4	Archivo de configuración del Reverse Proxy WAF (nginx_waf.conf)	Script	Mauricio Martínez	Repositorio GitHub / nginx_waf.conf
5	Captura de consola mostrando la respuesta **HTTP 403 Forbidden** en la prueba de ráfagas	Captura de pantalla	Mauricio Martínez	Repositorio de GitHub: prueba final WAF protegiendo sitio web local.png
6	Script ejecutable en gcloud CLI (deploy_waf_policy.sh) para desplegar en Cloud Armor	Script	Mauricio Martínez	Repositorio GitHub / deploy_waf_policy.sh

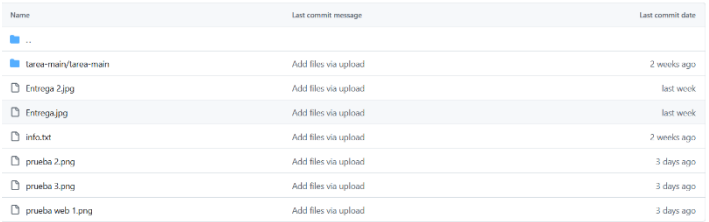
Criterio de calidad de la evidencia: La evidencia debe permitir verificar el aporte sin ambigüedad. Se recomienda incluir identificadores concretos: URL, commit hash, nombre de archivo, ruta de carpeta, nombre de prueba, nombre de servicio, fecha y referencia del estudiante responsable.

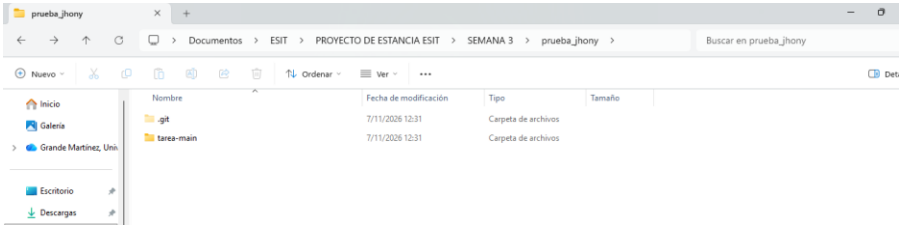
15.Registro de evidencias

Descripción de la evidencia:

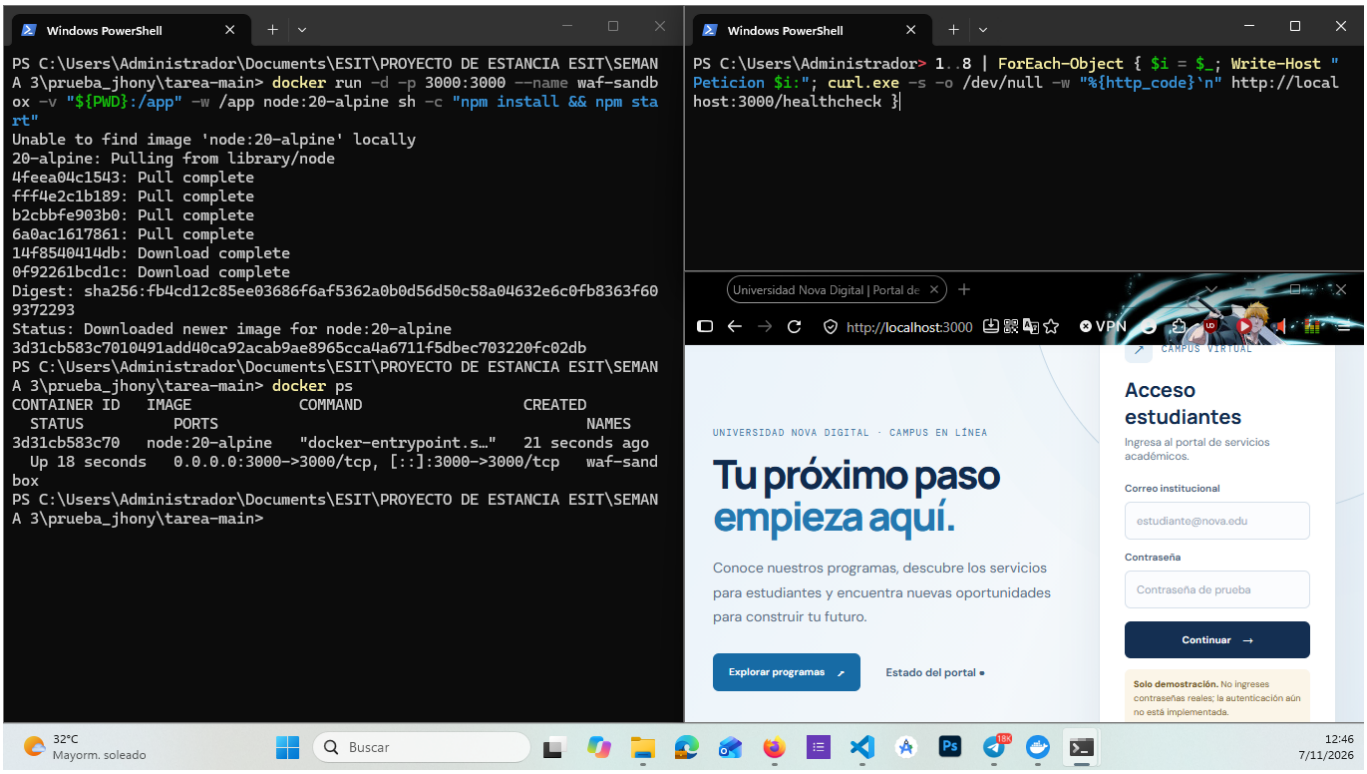
Evidencia 1: Captura de la clonación local y despliegue del entorno web del desarrollador Jhony Jiménez dentro del Sandbox aislado para la ejecución de las pruebas de auditoría y línea base.

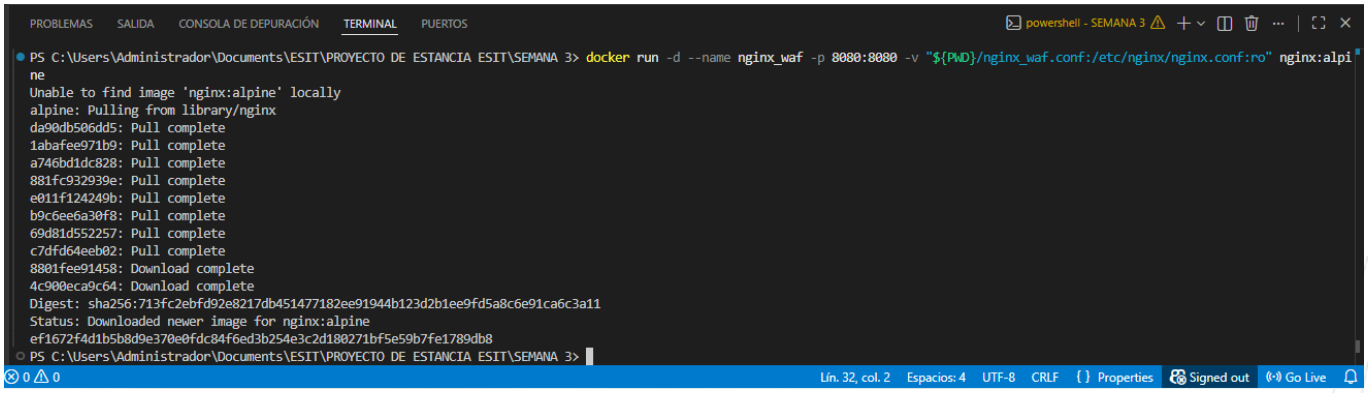






Evidencia 2: Captura de pantalla de la terminal de comandos confirmando la ejecución activa de los contenedores Docker (waf-sandbox en el puerto 3000 y nginx_waf en el puerto 8080).





Evidencia 3: Captura del código fuente del script automatizado en Python (test_waf_mitigated.py) utilizado para auditar la suite de peticiones HTTP y evaluar los vectores de ataque.

Windows PowerShell

```
PS C:\Users\Administrador> cd "C:\Users\Administrador\Documents\ESIT\PROYECTO DE ESTANCIA ESIT\SEMANA 3"
PS C:\Users\Administrador\Documents\ESIT\PROYECTO DE ESTANCIA ESIT\SEMANA 3> python test_baseline_waf.py
================================================================================
 AUDITORÍA SECLINE DE LÍNEA BASE PERIMETRAL - GRUPO SN-2
================================================================================

--- 1. SALUD DEL SERVICIO (GET /healthcheck) ---
Estado: StatusCode = 200 -> Servicio Operativo Base

--- 2. PRUEBA DE FUERZA BRUTA / RATE LIMITING (POST /login) ---
Peticion 1: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 2: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 3: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 4: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 5: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 6: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 7: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 8: StatusCode = 501 (Procesada por Express sin Rate Limit)

--- 3. SIMULACIÓN DE INYECCIÓN SQL - SQLi (POST /login) ---
Inyeccion SQL: StatusCode = 501 (Llega al backend -> Mitigar con Regla 2000)

--- 4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS (GET /search) ---
XSS Reflejado en Query: '<script>alert('XSS-Test')</script>' -> StatusCode = 200 (Sin filtrado -> Mitigar con Regla 3000)

--- 5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO (POST /comments) ---
Comentario: StatusCode = 201 (Llega a la app -> Mitigar con Regla 2000/3000)

--- 6. VERIFICACIÓN DE RUTA RESTRINGIDA (GET /admin) ---
Ruta Admin: StatusCode = 403 (Restringida por backend -> Regla 4000 de respaldo)

================================================================================
 FIN DE LA AUDITORÍA DE LÍNEA BASE - EVIDENCIA LISTA PARA DOCUMENTACIÓN
================================================================================
```

SIN WAF

Windows PowerShell

```
PS C:\Users\Administrador\Documents\ESIT\PROYECTO DE ESTANCIA ESIT\SEMANA 3> python test_waf_mitigated.py
================================================================================
 AUDITORÍA SECLINE DE LÍNEA BASE PERIMETRAL - GRUPO SN-2
================================================================================

--- 1. SALUD DEL SERVICIO (GET /healthcheck) ---
Estado: StatusCode = 200 -> Servicio Operativo Base

--- 2. PRUEBA DE FUERZA BRUTA / RATE LIMITING (POST /login) ---
Peticion 1: StatusCode = 501 (Procesada por Express sin Rate Limit)
Peticion 2: StatusCode = 403 (Procesada por Express sin Rate Limit)
Peticion 3: StatusCode = 403 (Procesada por Express sin Rate Limit)
Peticion 4: StatusCode = 403 (Procesada por Express sin Rate Limit)
Peticion 5: StatusCode = 403 (Procesada por Express sin Rate Limit)
Peticion 6: StatusCode = 403 (Procesada por Express sin Rate Limit)
Peticion 7: StatusCode = 403 (Procesada por Express sin Rate Limit)
Peticion 8: StatusCode = 403 (Procesada por Express sin Rate Limit)

--- 3. SIMULACIÓN DE INYECCIÓN SQL - SQLi (POST /login) ---
Inyeccion SQL: StatusCode = 403 (Llega al backend -> Mitigar con Regla 2000)

--- 4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS (GET /search) ---
XSS Reflejado en Query: '<script>alert('XSS-Test')</script>' -> StatusCode = 200 (Sin filtrado -> Mitigar con Regla 3000)

--- 5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO (POST /comments) ---
Comentario: StatusCode = 201 (Llega a la app -> Mitigar con Regla 2000/3000)

--- 6. VERIFICACIÓN DE RUTA RESTRINGIDA (GET /admin) ---
Ruta Admin: StatusCode = 403 (Restringida por backend -> Regla 4000 de respaldo)

================================================================================
 FIN DE LA AUDITORÍA DE LÍNEA BASE - EVIDENCIA LISTA PARA DOCUMENTACIÓN
================================================================================

PS C:\Users\Administrador\Documents\ESIT\PROYECTO DE ESTANCIA ESIT\SEMANA 3>
```

CON WAF

```python
# ================================================================================
# SCRIPT DE AUDITORÍA Y LÍNEA BASE PERIMETRAL - WAF SHIELD CLOUD (GRUPO SN-2)
# Entorno: Docker Sandbox Local (http://localhost:8080)
# ================================================================================

import requests
import time

BASE_URL = "http://localhost:8080"

def print_header(title):
    print(f"\n--- {title} ---")

print("================================================================================")
print(" AUDITORÍA SECLINE DE LÍNEA BASE PERIMETRAL - GRUPO SN-2")
print("================================================================================")

# 1. SALUD DEL SERVICIO
print_header("1. SALUD DEL SERVICIO (GET /healthcheck)")
try:
    r = requests.get(f"{BASE_URL}/healthcheck")
    print(f"Estado: StatusCode = {r.status_code} -> Servicio Operativo Base")
except Exception as e:
    print(f"Estado: Error al conectar ({e})")

# 2. PRUEBA DE FUERZA BRUTA / RATE LIMITING
print_header("2. PRUEBA DE FUERZA BRUTA / RATE LIMITING (POST /login)")
payload_login = {"email": "demo@example.test", "password": "123"}
for i in range(1, 9):
    try:
        r = requests.post(f"{BASE_URL}/login", json=payload_login)
        print(f"Peticion {i}: StatusCode = {r.status_code} (Procesada por Express sin Rate Limit)")
    except Exception as e:
        print(f"Peticion {i}: Error ({e})")

# 3. SIMULACIÓN DE INYECCIÓN SQL - SQLi
print_header("3. SIMULACIÓN DE INYECCIÓN SQL - SQLi (POST /login)")
payload_sqli = {"email": "admin' OR '1'='1", "password": "123"}
try:
    r = requests.post(f"{BASE_URL}/login", json=payload_sqli)
    print(f"Inyeccion SQL: StatusCode = {r.status_code} (Llega al backend -> Mitigar con Regla 2000)")
except Exception as e:
    print(f"Inyeccion SQL: Error ({e})")

# 4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS
print_header("4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS (GET /search)")
try:
    r = requests.get(f"{BASE_URL}/search", params={"q": "<script>alert('XSS-Test')</script>"})
    if r.status_code == 200:
        data = r.json()
        print(f"XSS Reflejado en Query: '{data.get('query')}' -> StatusCode = 200 (Sin filtrado -> Mitigar con Regla 3000)")
    else:
        print(f"XSS: StatusCode = {r.status_code}")
except Exception as e:
    print(f"XSS: Error ({e})")

# 5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO
print_header("5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO (POST /comments)")
payload_xss_comment = {"comment": "<script>alert('xss')</script>"}
try:
    r = requests.post(f"{BASE_URL}/comments", json=payload_xss_comment)
    print(f"Comentario: StatusCode = {r.status_code} (Llega a la app -> Mitigar con Regla 2000/3000)")
except Exception as e:
    print(f"Comentario: Error ({e})")

# 6. VERIFICACIÓN DE RUTA RESTRINGIDA
print_header("6. VERIFICACIÓN DE RUTA RESTRINGIDA (GET /admin)")
try:
    r = requests.get(f"{BASE_URL}/admin")
    print(f"Ruta Admin: StatusCode = {r.status_code} (Restringida por backend -> Regla 4000 de respaldo)")
except Exception as e:
    print(f"Ruta Admin: Error ({e})")

print("\n================================================================================")
print(" FIN DE LA AUDITORÍA DE LÍNEA BASE - EVIDENCIA LISTA PARA DOCUMENTACIÓN")
print("================================================================================\n")

```

Evidencia 4: Captura del archivo de configuración del Reverse Proxy Nginx (nginx_waf.conf) detallando las directivas de estrangulamiento de tasa (limit_req_zone) y denegación de rutas.

```
events {
    worker_connections 1024;
}

http {
    limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;

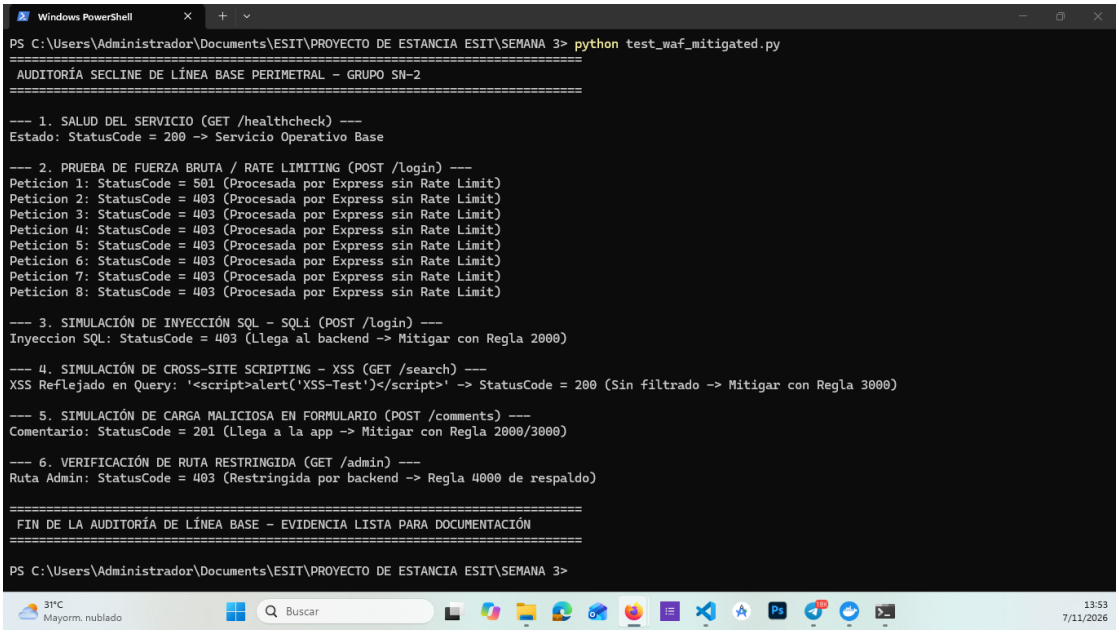
    server {
        listen 8080;

        # Regla 1000: Rate Limiting en /login (Sin ráfaga / nodelay)
        location /login {
            limit_req zone=login_limit nodelay;
            limit_req_status 403;
            proxy_pass http://host.docker.internal:3000;
        }

        # Regla 4000: Bloqueo de /admin
        location /admin {
            return 403;
        }

        # Tráfico general (Default Allow)
        location / {
            proxy_pass http://host.docker.internal:3000;
        }
    }
}
```

Evidencia 5: Captura de la terminal de PowerShell mostrando la respuesta empírica **HTTP 403 Forbidden** ante el intento de ráfagas en el formulario de /login.



Evidencia 6: Captura del script ejecutable en Bash (deploy_waf_policy.sh) diseñado para automatizar el despliegue de la política perimetral en Google Cloud Armor (gcloud CLI).



```bash
#!/bin/bash
# ------------------------------------------------------------------------------
# PROYECTO: WAF Shield Cloud - Grupo SN-2
# DESCRIPCIÓN: Script de despliegue automatizado de política perimetral en GCP
# AUTOR: Mauricio Alberto Grande Martinez
# ------------------------------------------------------------------------------

# 1. Crear la política de seguridad perimetral en Cloud Armor
gcloud compute security-policies create waf-shield-sn2-policy \
    --description="Politica perimetral WAF Shield Cloud - Grupo SN-2"

# 2. Regla 1000: Rate Limiting en /login (Mitigación de Fuerza Bruta)
# Restringe a máximo 5 peticiones por minuto por dirección IP; banea por 5 minutos al exceder.
gcloud compute security-policies rules create 1000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="request.path.matches('/login')" \
    --action="rate-based-ban" \
    --rate-limit-threshold-count=5 \
    --rate-limit-threshold-interval-sec=60 \
    --ban-duration-sec=300 \
    --conform-action="allow" \
    --exceed-action="deny(403)" \
    --enforce-on-key="IP"

# 3. Regla 2000: Protección contra Inyección SQL (SQLi en L7)
# Utiliza el conjunto de reglas preconfiguradas de OWASP para SQLi
gcloud compute security-policies rules create 2000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="evaluatePreconfiguredExpr('sqli-v933100')" \
    --action="deny(403)"

# 4. Regla 3000: Protección contra Cross-Site Scripting (XSS en L7)
# Utiliza el conjunto de reglas preconfiguradas de OWASP para XSS
gcloud compute security-policies rules create 3000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="evaluatePreconfiguredExpr('xss-v941100')" \
    --action="deny(403)"

# 5. Regla 4000: Restricción de acceso a rutas administrativas (/admin)
# Intercepta en el perímetro cualquier intento de acceso a la consola interna
gcloud compute security-policies rules create 4000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="request.path.matches('/admin')" \
    --action="deny(403)"
```